Экспериментальное исследование алгоритмов на основе линейной алгебры

LAGraph и SPLA

Мелашенко Ксения, Б-11

Постановка задачи

Цель эксперимента – сравнить время работы реализаций графовых алгоритмов на основе линейной алгебры из библиотек **Spla (на GPU)** и **LAGraph (на CPU)**

Исследуемые алгоритмы:

- **BFS** (Breadth-first search)
- **TC** (Triangle Counting)
- **SSSP** (Single-Source Shortest Path)
- **PR** (PageRank)

Оборудование

- **CPU**: Intel i7-12700H, 6 P-cores (12 threads) + 8 E-cores
- **RAM**: 32GB
- **GPU**: NVIDIA T1200 Laptop, 4 GB
- **OS**: Fedora 42, kernel 6.17.9
- **opencl-nvidia**: 580.105.08
- **OpenCL**: 3.0

- **LAGraph**: v1.2.2
- **Spla**: commit b81d1e2

Описание набора данных для экспериментов

Name	|V|	|E|	avg Deg
coAuthorsCiteseer	227K	1.6M	7.2
roadNet-CA	2.0M	5.5M	2.8
belgium_osm	1.44M	3.1M	2.2
delaunay_n21	2.1M	12.6M	6.0
coPapersDBLP	540K	30.5M	56.4
road_central	14.1M	33.9M	2.4
rgg_n_2_22_s0	4.2M	60.7M	14.5
hollywood-2009	1.1M	112.8M	98.9
com-Orkut	3.1M	234.4M	76.3

Все графы – неориентированные

Описание эксперимента

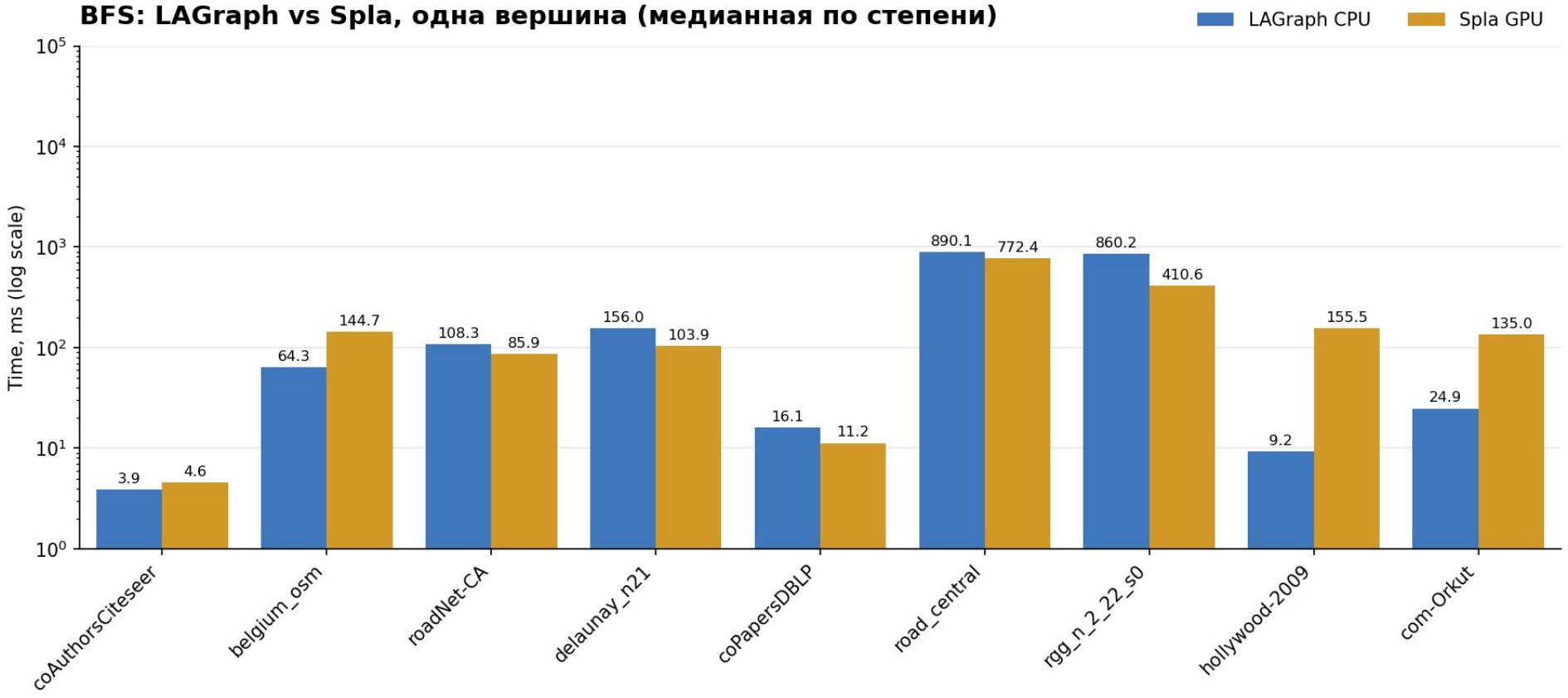
Метрика времени

Медиана по запускам алгоритма 2 … N раз (N = 20). Первый запуск является прогревом и в статистику не входит.

Выбор стартовых вершин для BFS и SSSP

Выбрана вершина с медианной степенью в наибольшей компоненте связности

Результаты BFS



Результаты BFS

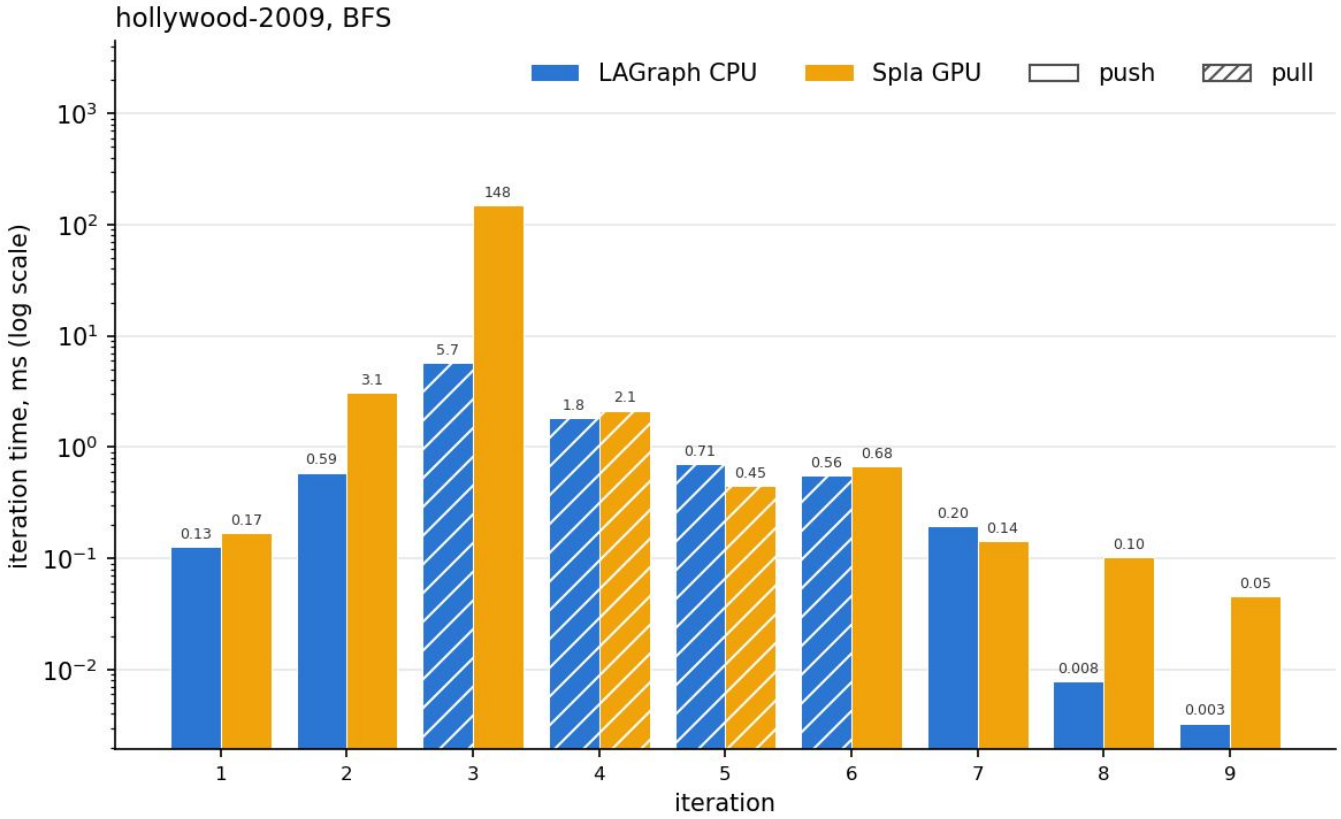
- SPLA выигрывает на графах:
  road_central, rgg_n_2_22_s0, roadNet-CA, delaunay_n21, coPapersDBLP

- LAGraph выигрывает на графах:
  belgium_osm, coAuthorsCiteseer, hollywood-2009, com-Orkut

	dataset	iters num	avg deg	avg frontier	avg edges per iter
*	road_central	3,582	2.4	3,931	9,455
	belgium_osm	1,600	2.2	901	1,937
*	rgg_n_2_22_s0	1,206	14.5	3,476	50,326
*	roadNet-CA	630	2.8	3,106	8,783
*	delaunay_n21	557	6.0	3,765	22,590
	coAuthorsCiteseer	25	7.2	9,093	65,131
*	coPapersDBLP	14	56.4	38,606	2,177,961
	hollywood-2009	8	99.9	133,641	14,236,416
	com-Orkut	7	76.3	438,920	33,481,452

Время итераций push/pull BFS

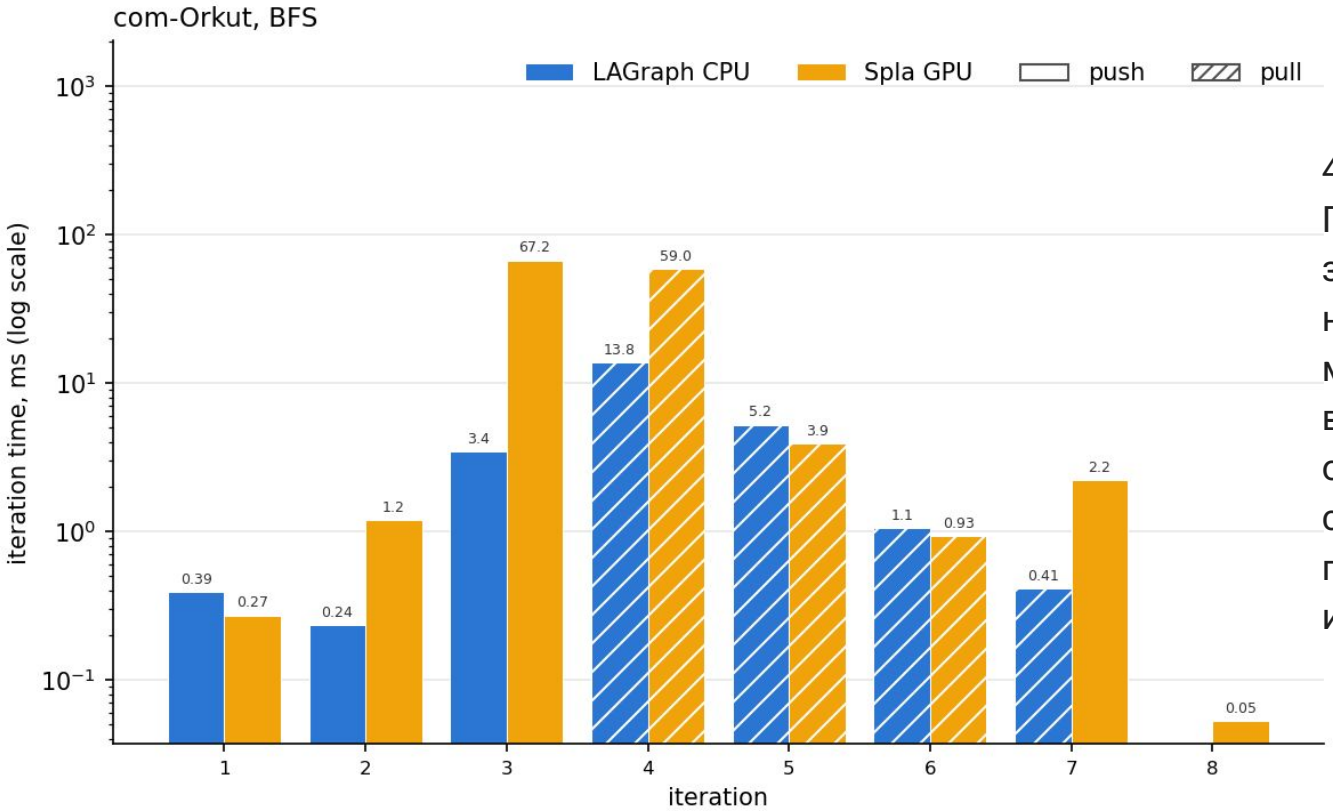
95% времени – одна операция push



3 итерация:
LAGraph уже перешёл в pull, а SPLA ещё в push (порог – доля вершин во фронте)

65% времени в push уходит на сортировку

Время итераций push/pull BFS



4 итерация:
Первая pull-итерация. GPU занят 98.7 % Из 2.36 млн непосещенных вершин 1.25 млн (53 %) не имеют соседа во фронте. Для них ранняя остановка не срабатывает, и строка читается целиком => главное преимущество pull исчезает

Вывод BFS

- Spla на GPU выигрывает, когда на каждой итерации достаточно полезной работы на GPU. Накладные расходы окупаются, выигрыш растет с числом рёбер на итерацию, а LAGraph начинает распределять большие вычисления на несколько ядер
- При малом фронте выигрывает LAGraph на CPU за счёт дешёвой итерации без накладных расходов на запуск и синхронизацию с GPU (belgium_osm, coAuthorsCiteceer).
- На графах com-Orkut и hollywood2009 почти все время работы сосредоточены в одной-двух итерациях. Основная причина на обоих графах — push через сортировку пар. Он дорог на фронтах с большим числом рёбер, а на hollywood-2009 это усугубляет порог переключения с push на pull

Алгоритм SSSP

SPLA SSSP использует то же умножение вектора на матрицу, как в BFS, только в полукольце (min, +)

LAGraph SSSP использует алгоритм delta-stepping. Значение delta (∆) влияет на то, какое множество вершин мы рассматриваем на данной итерации, чтоб релаксировать их ребра. Такое множество называется корзиной.

- При единичных весах и и Δ=1.0 (для LAGraph) SSSP проходит граф по тем же уровням, что и BFS. Результаты алгоритма и число итераций совпадают.

- BFS использует логическое полукольцо (or, and), а SSSP — (min, +)

Delta-stepping

Основная идея: разбивать вершины по их расстояниям t на корзины и последовательно обрабатывать корзины, а вершины внутри одной корзины — одновременно.

- **Расстояния** `t[v]` – длина самого короткого пути от стартовой вершины до вершины v. В начале `t[source] = 0`, остальные равны ∞
- **Корзина k** — вершины, входящие в диапазон $k\Delta \leq t[v] < (k+1)\Delta$, где `k` – это номер текущей корзины

Delta-stepping

Ребра делятся по весу:

- Легкие ($w \leq \Delta$): могут вернуть в текущую корзину
- Тяжелые ($w > \Delta$): всегда ведут в следующие корзины

Обработка одной корзины:

1. Собрать корзину
2. Релаксировать легкие ребра из корзины (умножить вектора на матрицу в полукольце (min, +)). Повторять, пока в корзине есть улучшающиеся вершины.
3. Релаксировать тяжёлые рёбра из всей корзины. Закрыть корзину. Ее расстояния окончательные

Сборка корзины в LAGraph для SSSP

Корзины нигде не хранятся. **В начале каждого шага корзина k заново вычисляется** из векторов

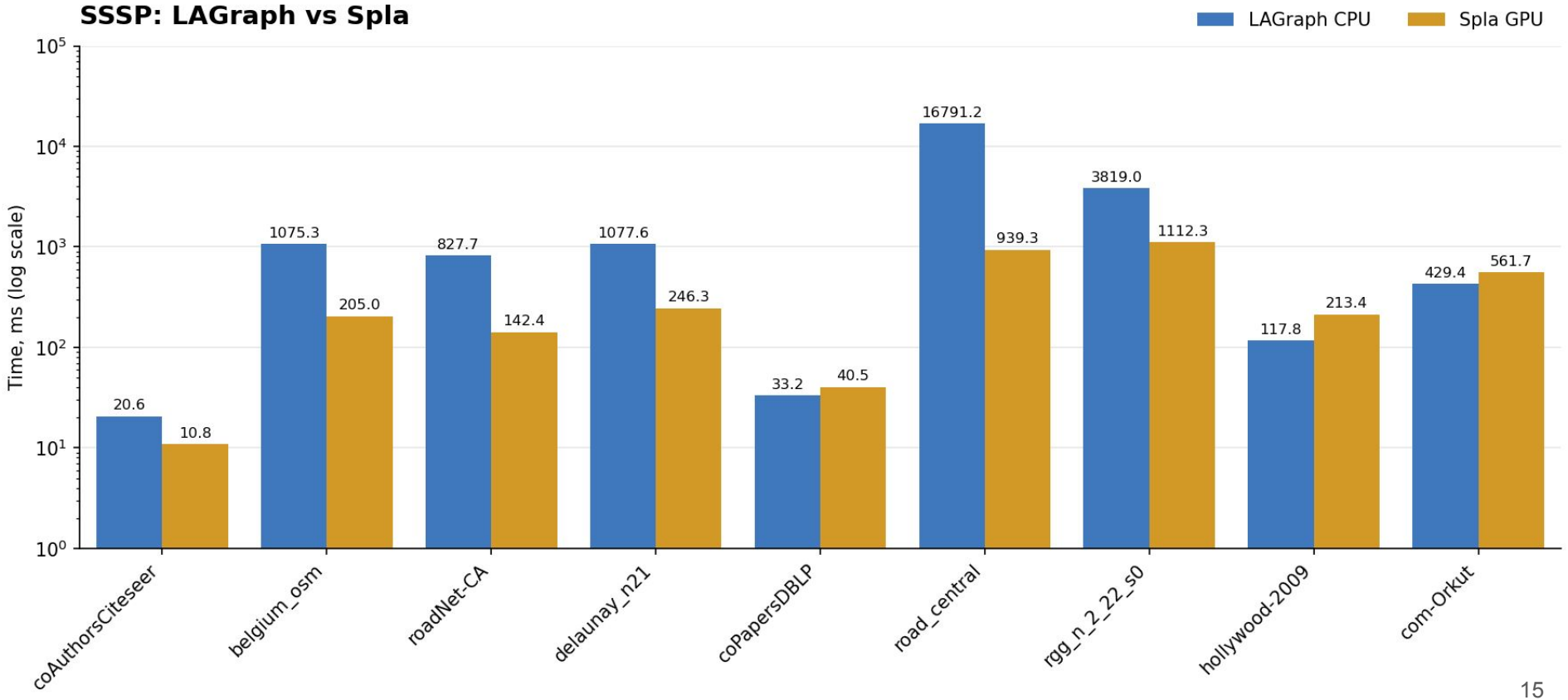
- `t:` вектор наикратчайших расстояний длиной n (∞ – ещё не найдена)
- `reach:` найденные вершины-кандидаты из предыдущих корзин в следующие корзины

Строится фронт, куда попадают значения из t с маской reach, а также дополнительно исключаются значения >= `(k+1)Δ`

Эта сборка стоит O(n) на каждую корзину, вне зависимости от размера корзины

Результаты SSSP

На длинных обходах выигрывает SPLA, так как по профилю perf большая часть времени SSSP LAGraph уходит на построение корзин

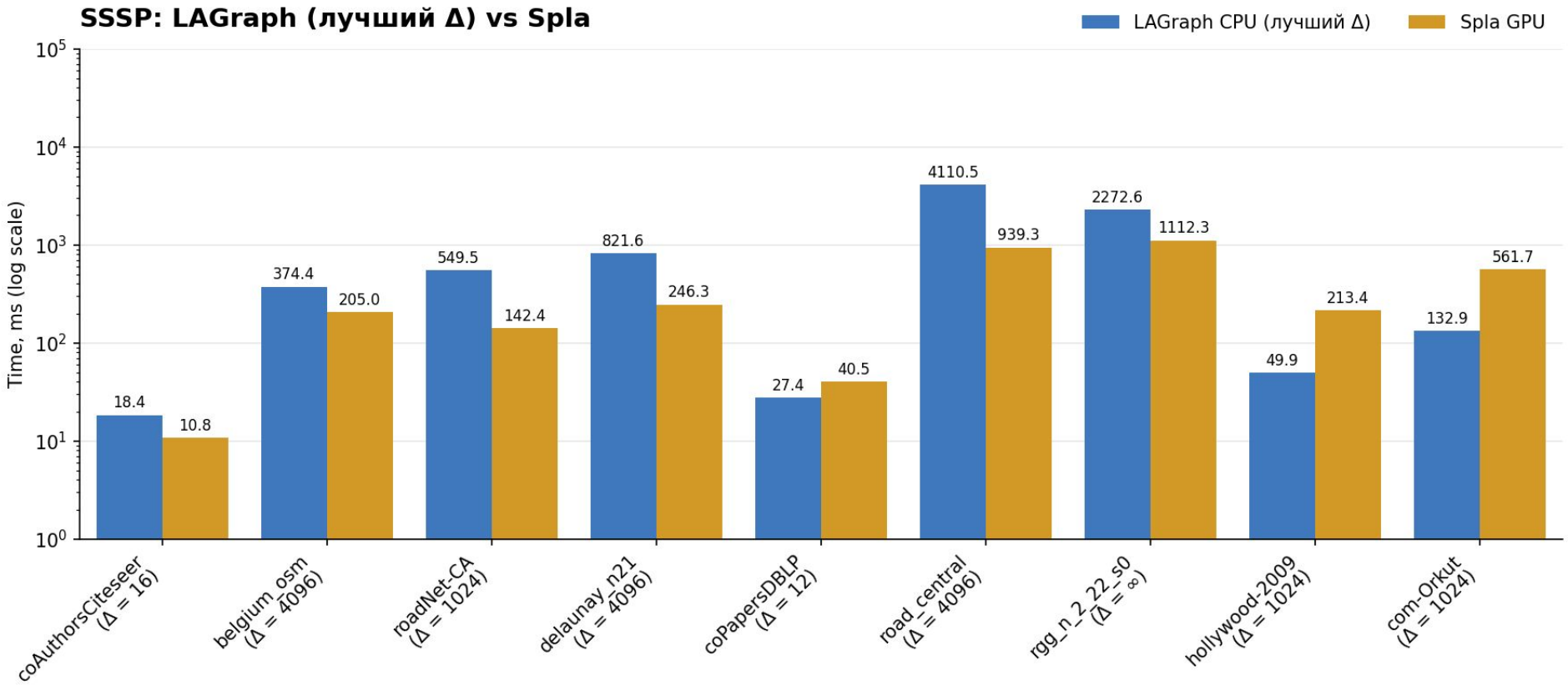


Результаты SSSP (зависимость от Δ)

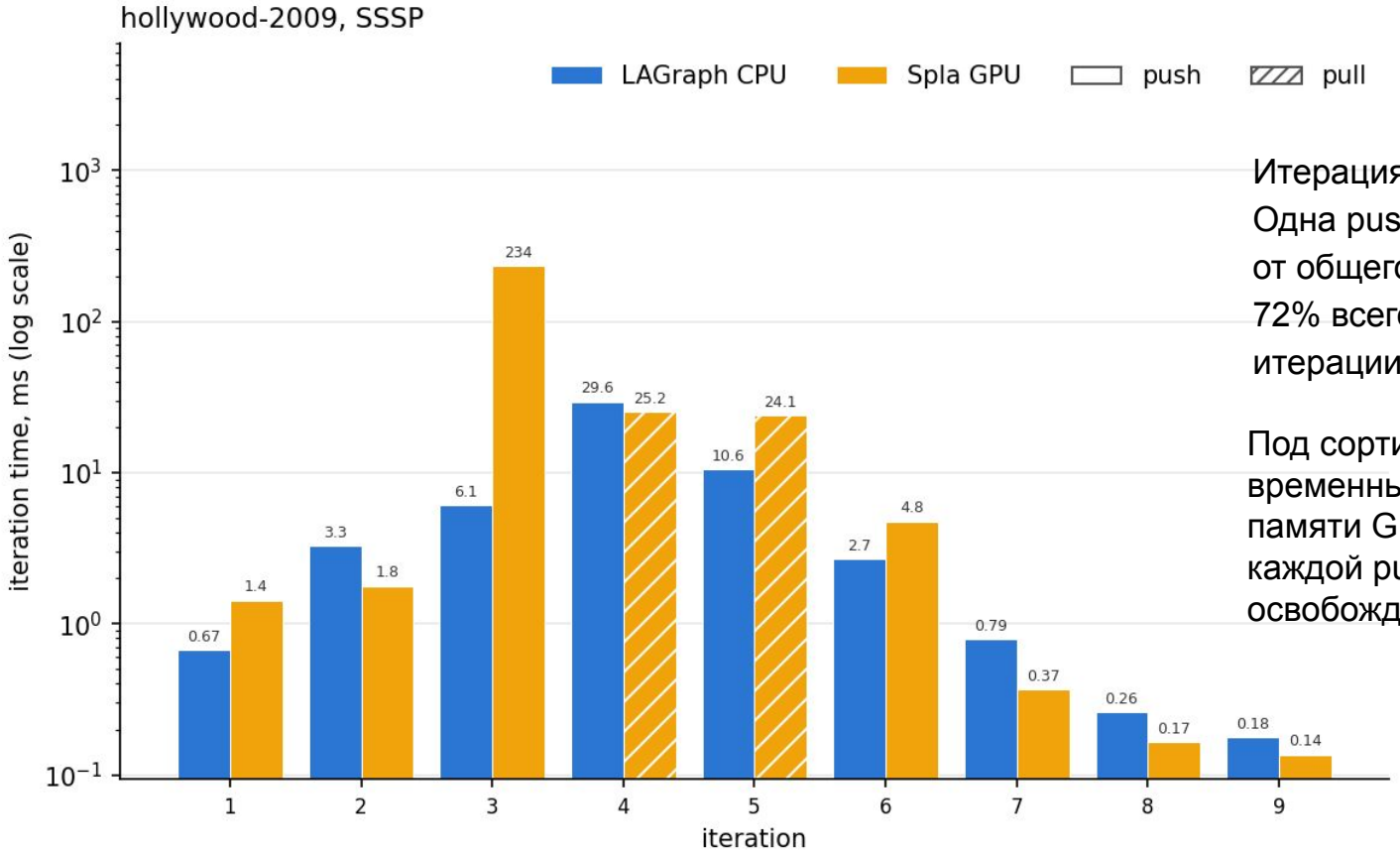
Δ	1	2	3	4	6	8	12	16	24	32	64	128	256	1024	4096	∞
coAuthorsCiteseer	1.00×	1.01×	1.03×	1.03×	1.04×	1.05×	1.07×	1.08×	1.06×	1.06×	1.07×	1.07×	1.06×	1.07×	1.06×	1.06×
coPapersDBLP	1.00×	1.02×	1.02×	1.04×	1.06×	1.09×	1.14×	1.14×	1.12×	1.14×	1.14×	1.14×	1.14×	1.14×	1.14×	1.14×
hollywood-2009	1.00×	1.06×	1.10×	1.17×	1.34×	1.32×	1.30×	1.32×	1.33×	1.32×	1.32×	1.33×	1.31×	1.34×	1.31×	1.32×
com-Orkut	1.00×	1.10×	1.23×	1.46×	1.62×	1.62×	1.64×	1.63×	1.63×	1.62×	1.61×	1.62×	1.65×	1.65×	1.64×	1.64×
belgium_osm	1.00×	1.22×	1.28×	1.34×	1.42×	1.46×	1.50×	1.52×	1.52×	1.56×	1.61×	1.71×	1.80×	2.15×	2.20×	2.20×
roadNet-CA	1.00×	1.06×	1.04×	1.05×	1.06×	1.07×	1.11×	1.11×	1.13×	1.15×	1.22×	1.28×	1.34×	1.44×	1.44×	1.43×
delaunay_n21	1.00×	1.02×	1.01×	1.01×	1.02×	1.04×	1.04×	1.05×	1.07×	1.08×	1.12×	1.15×	1.19×	1.25×	1.25×	1.25×
rgg_n_2_22_s0	1.00×	1.09×	1.13×	1.15×	1.17×	1.19×	1.20×	1.21×	1.21×	1.22×	1.26×	1.30×	1.34×	1.51×	1.54×	1.54×
road_central	1.00×	1.51×	1.81×	1.96×	2.13×	2.21×	2.31×	2.37×	2.47×	2.55×	2.72×	2.83×	2.91×	3.09×	3.55×	3.54×

- Δ в LAGraph влияет только на число корзин. При единичных весах delta-stepping не делает лишних вычислений.
- Подобранный Δ ускоряет LAGraph, сильнее всего на road_central, где больше всего итераций (4 638) и вершин (14 млн).
- Дальнейшее увеличение Δ после оптимального почти ничего не меняет. На плотных графах это наступает уже при Δ = 8-16 (8-25 уровней), на глубоких при Δ = 1024 (road_central при Δ = 4096).

Результаты SSSP



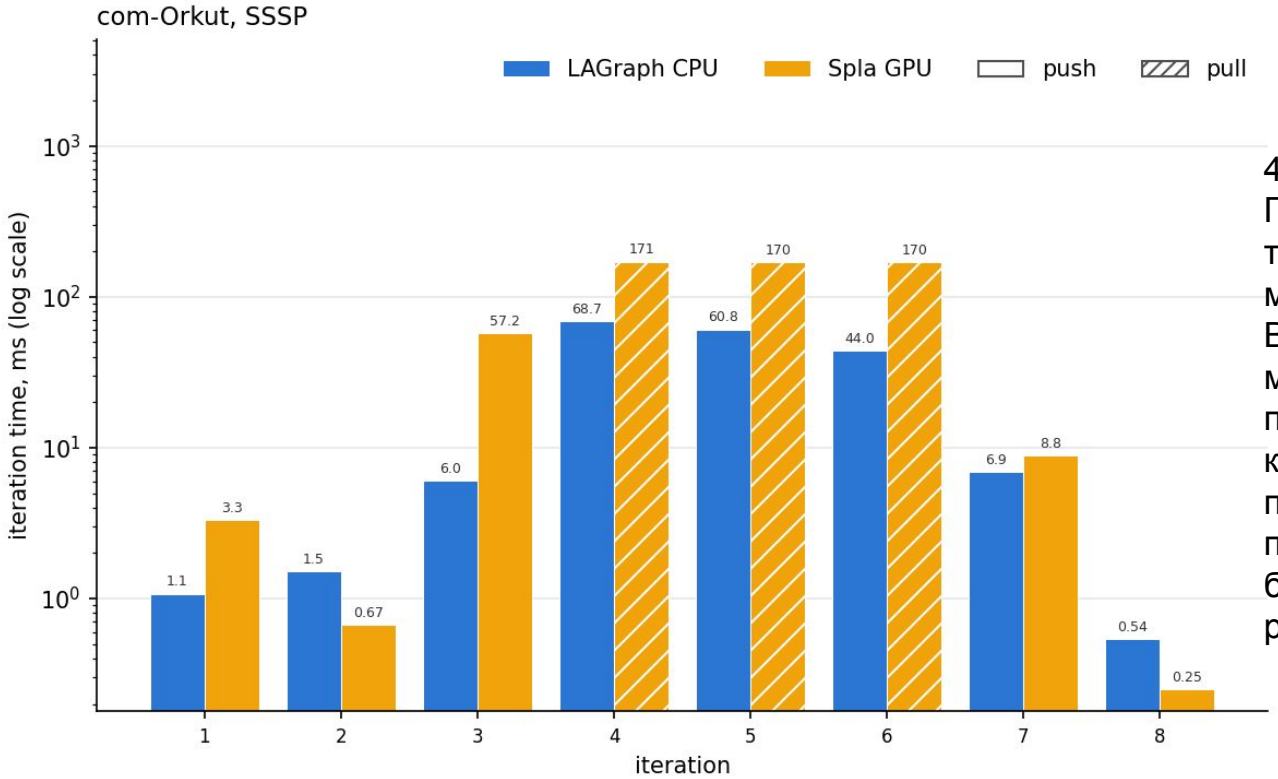
Время итераций push/pull SSSP



Итерация 3.
Одна push итерация занимает 80% от общего времени выполнения.
72% всего времени push на 3 итерации – сортировка

Под сортировку выделяются временные массивы с парами в памяти GPU. Они создаются на каждой push-итерации и освобождаются в её конце.

Время итераций push/pull SSSP



4-6 итерация.
Почти весь проигрыш из-за pull: трижды читает все строки в матрице.
В полукольце (min, +) нужен минимум по всем соседям, поэтому нет ранней остановки, как в BFS. К тому же, здесь не применяется маска для посещенных вершин, как это было в BFS, так как важны расстояния всех ребер

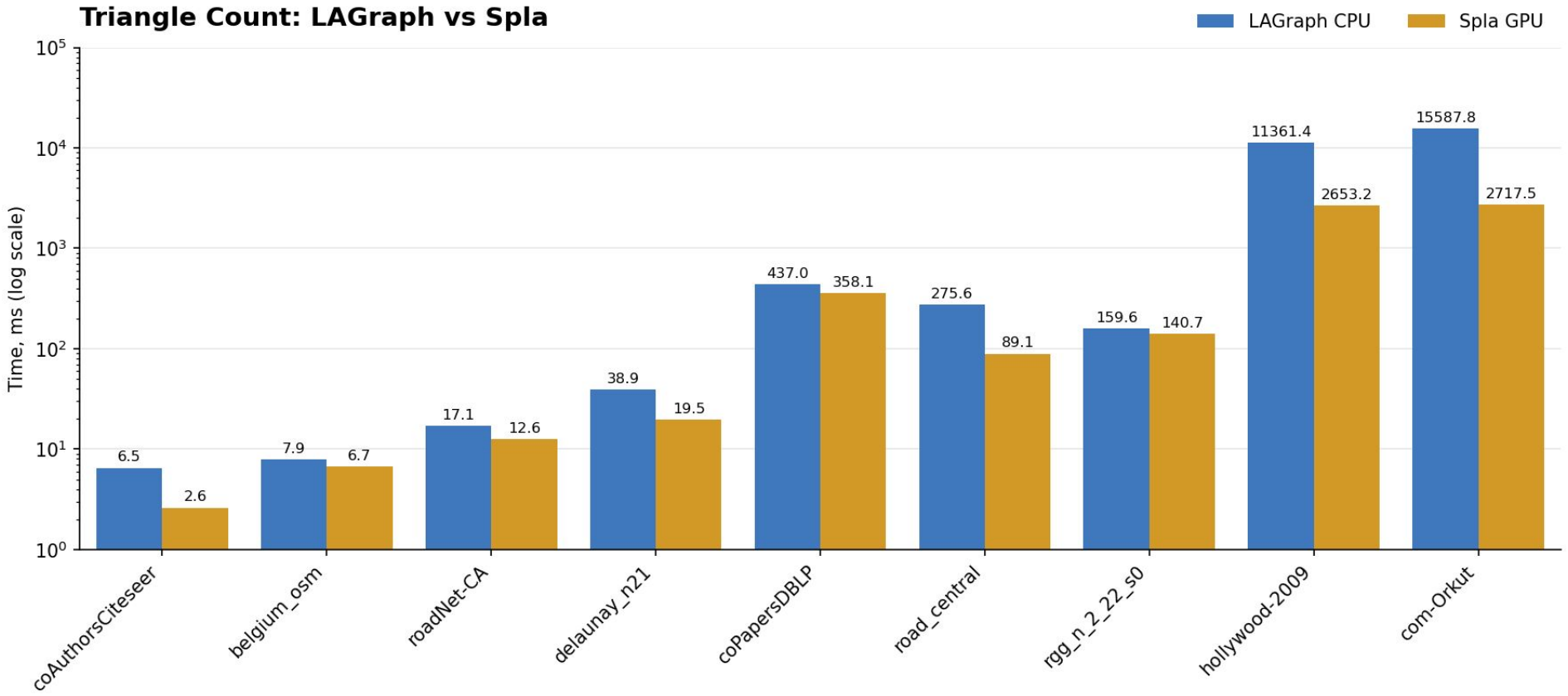
Вывод SSSP

1. Шаг LAGraph заново собирает корзину на каждую итерацию и эта сборка стоит O(n) на каждую корзину, вне зависимости от размера корзины. На графах, где с выбранной Δ попадает малое число вершин в корзину, сборка корзины на одной итерации занимает больше, чем полезная работа.

2. Δ в LAGraph влияет только на число корзин. При единичных весах delta-stepping не делает лишних вычислений

3. На плотных графах быстрее LAGraph, причина в операциях SPLA

Результаты TC



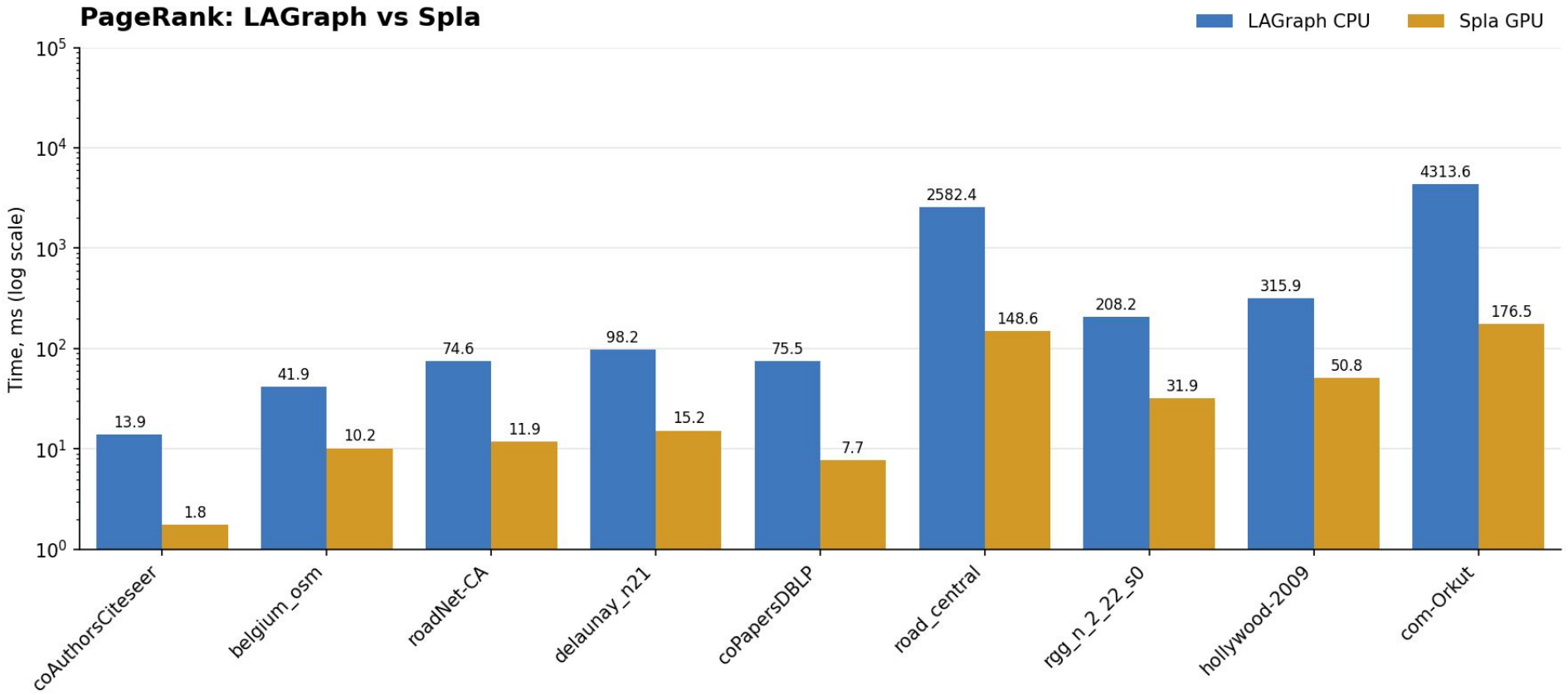
LAGraph: Sandia_LL , NoSort

Результаты TC

SPLA выигрывает на всех графах.

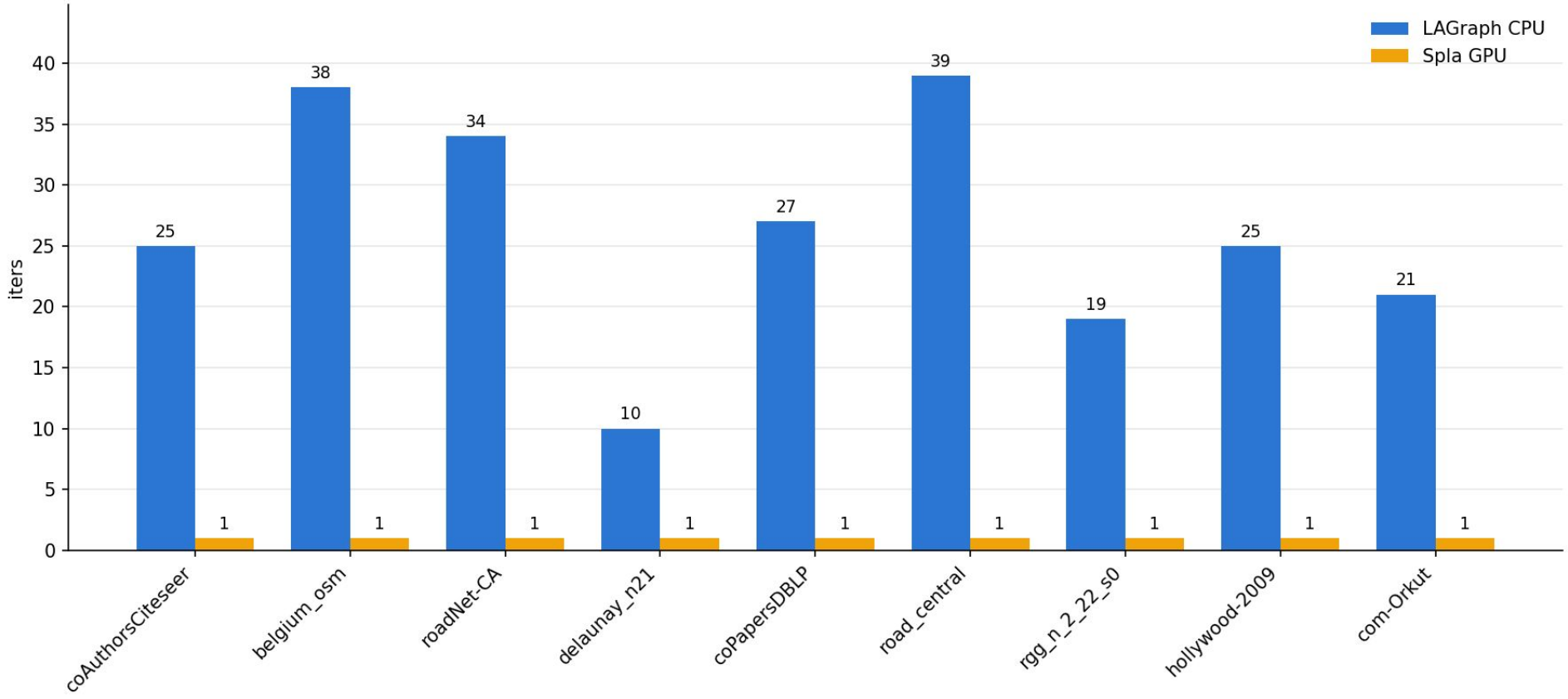
- В обеих библиотеках используется алгоритм Sandia: $\sum L \cdot (L \times L)$
- В TC нет итераций, так как используется только умножение матриц (`exec_mxmT_masked`) и последующее суммирование результата (`exec_m_reduce`).
- По диагностическим логам SPLA умножение занимает 73–91 % времени установившегося прогона

Результаты PR



alpha=0.85, eps=1e-4

Число итераций PR



Результаты PR

Выигрыш в производительности у SPLA на GPU наблюдается на **всех** графах

- LAGraph реализует алгоритм PR по итерационной математической формуле, где самая тяжелая операция – умножение матрицы на вектор

- SPLA: В PR каждая вершина делит свой ранг поровну между рангами соседей. В SPLA вершина вместо этого получает среднее рангов своих соседей. Поэтому у SPLA ранги после первой итерации не меняются, и она останавливается, возвращая всем вершинам одинаковый ранг.